

DAY-6

```
jupyter DAY-6 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Trusted

[1]: # 1.MEAN (Average)
##### The mean is calculated by adding all values and dividing by the total num of values.

[2]: #Average(mean)
marks=[45,90,80,35,20]
mean=sum(marks)/len(marks)
print("Mean:",mean)

Mean: 54.0

[3]: #Median
import statistics
marks=[45,90,80,35,20]
median=statistics.median(marks)
print("Median:",median)

Median: 45

[4]: #Mode
import statistics
marks=[45,90,80,35,20]
mode=statistics.mode(marks)
print("Mode:",mode)

Mode: 45

[5]: import pandas as pd
df=pd.DataFrame({'Marks': [45,90,80,35,20]
\`
```

```
jupyter DAY-6 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Trusted

[6]: print("Mean:")
print(df['Marks'].mean())
print("\nMedian:")
print(df['Marks'].median())
print("\nMode:")
print(df['Marks'].mode())

Mean:
54.0

Median:
45.0

Mode:
0    20
1     35
2     45
3     80
4     90
Name: Marks, dtype: int64

[7]: #variance
import numpy as np
marks=[45,90,80,35,20]
mean=np.mean(marks)
print("Mean:",mean)

Mean: 54.0

[8]: import numpy as np
marks=[45,90,80,35,20]
variance=np.var(marks)
print("Variance:",variance)

Variance: 714.0
```

```
jupyter DAY-6 Last Checkpoint: 5 hours ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Trusted

[9]: #Standard deviation
import numpy as np
marks=[45,90,80,35,20]
std=np.std(marks)
print("Standard deviation:",std)

Standard deviation: 26.720778431774775

[10]: #IQR code
import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
print("Q1:",Q1)
print("Q3:",Q3)
print("IQR:",IQR)

Q1: 13.75
Q3: 31.25
IQR: 17.5

[11]: import numpy as np
marks=[45,90,80,35,20]
Q1=np.percentile(marks,25)
Q3=np.percentile(marks,75)
IQR=Q3-Q1
Lowerlimit=Q1-1.5*IQR
Upperlimit=Q3+1.5*IQR
print(Lowerlimit)
print(Upperlimit)

Lowerlimit: -11.25
Upperlimit: 58.75
```

```
jupyter DAY-6 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base]

print(Upperlimit)
Outliers=[x for x in marks if x < Lowerlimit or x > Upperlimit]
print("Outliers",Outliers)

-32.5
147.5
Outliers []

[15]: #Probability Basics
sample=(1,2,3,4,5)
event=(2,4,6)
print("Sample space:",sample)
print("Event A:",event)
P_A=len(event)/len(sample)
print("\nProbability of Event A")
print("P(A)=",P_A)

Sample space: (1, 2, 3, 4, 5)
Event A: (2, 4, 6)

Probability of Event A
P(A)= 0.6

[16]: #Conditional Probability
event_B=(4,5,6)
inter=event.intersection(event_B)
P_A_given_B=len(inter)/len(event_B)
print("\nEvent B:",event_B)
print("A n B:", inter)
print("P(A|B)=", P_A_given_B)

Event B: (4, 5, 6)
```

```
jupyter DAY-6 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base]

A n B: (4, 6)
P(A|B)= 0.6666666666666666

[17]: #Bayes' Theora
P_Disease=0.01
P_Positive_Disease=0.99
P_Positive=0.05
P_Disease_given_Positive=(
    P_Positive_Disease*P_Disease
)/P_Positive
print("\n Bayes' Theora Example")
print("P(Disease)=",P_Disease)
print("P(Positive|Disease)=",P_Positive_Disease)
print("P(Positive)=",P_Positive)
print(
    "P(Disease|Positive)=",round(P_Disease_given_Positive,4)
)
print(
    "Percentage=",
    round(P_Disease_given_Positive * 100,2),"%"
)

Bayes' Theora Example
P(Disease)= 0.01
P(Positive|Disease)= 0.99
P(Positive)= 0.05
P(Disease|Positive)= 0.198
Percentage= 19.8 %

[18]: import pandas as pd
dfapd.DataFrame([
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,60,70]
])
```

```
jupyter DAY-6 Last Checkpoint: 5 hours ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base]

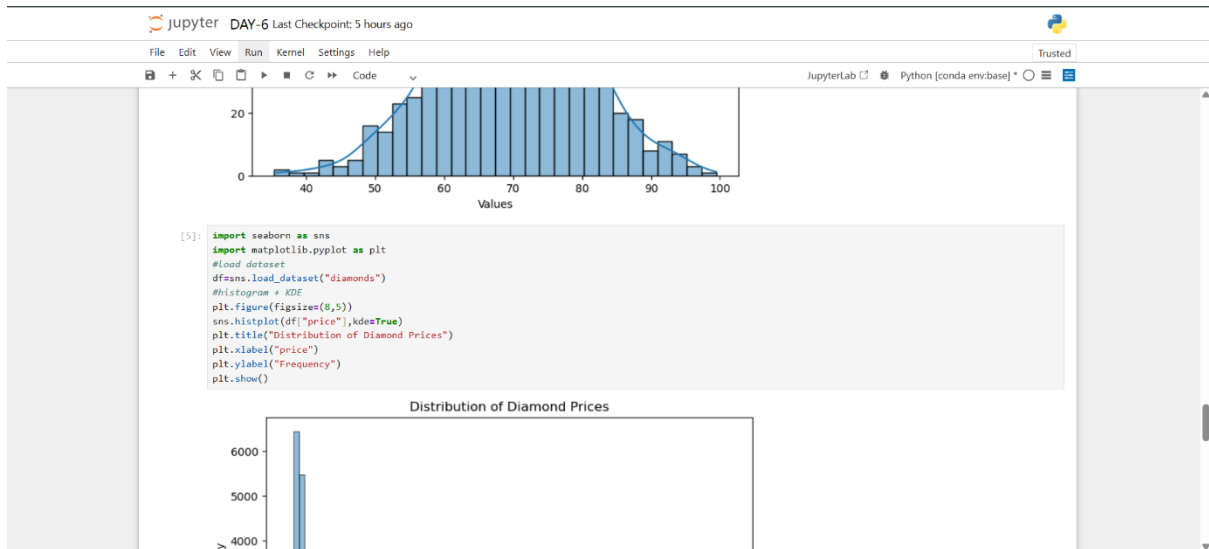
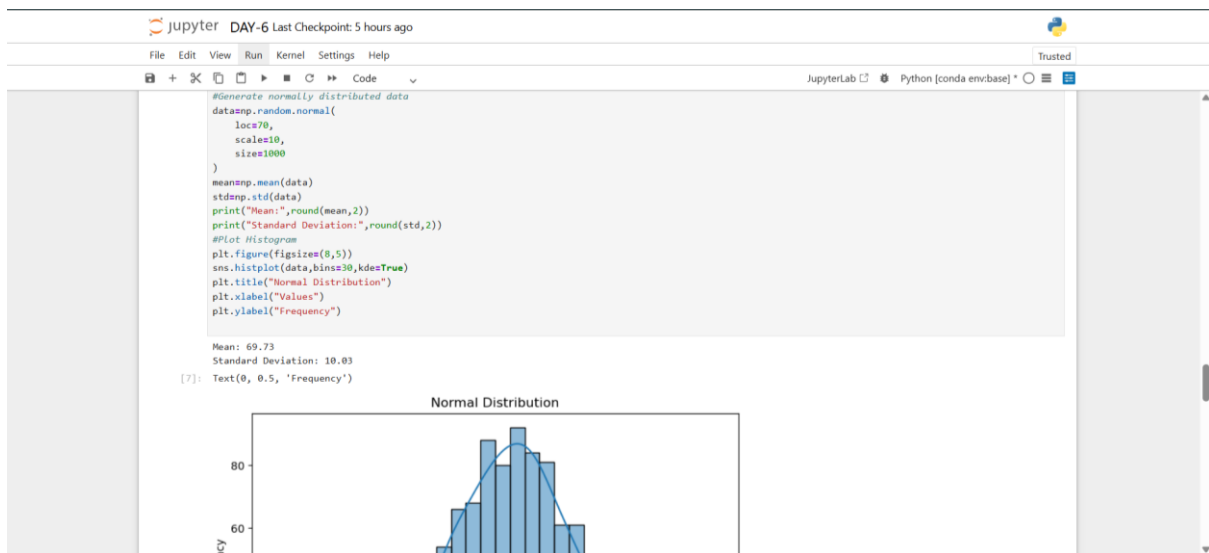
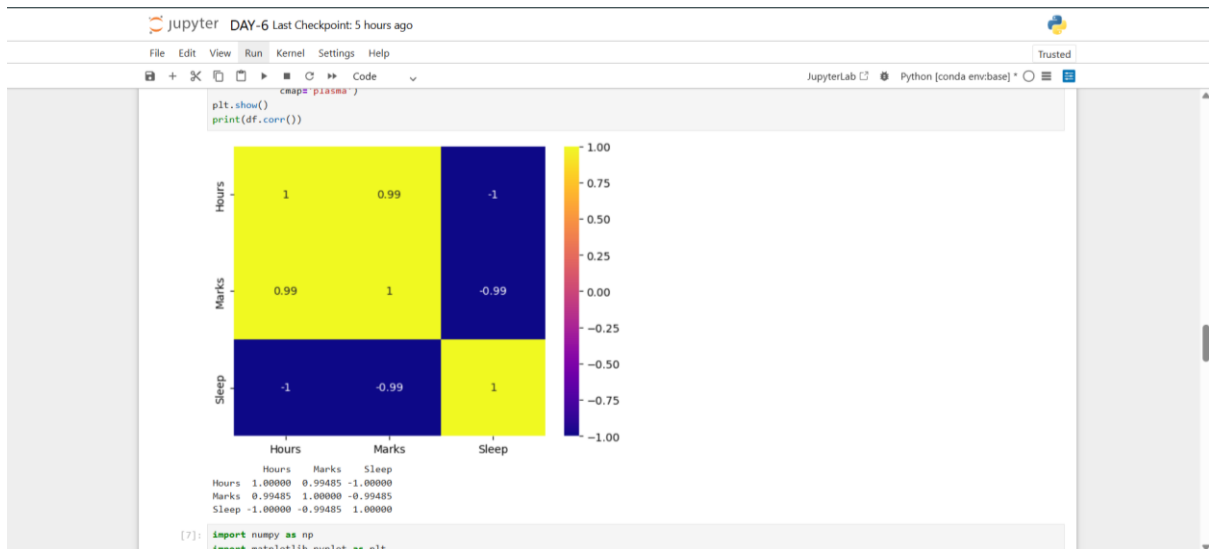
))
corr=df['Hours'].corr(df['Marks'])
print("Pearson correlation:",corr)
#close to 1
#strong positive relation

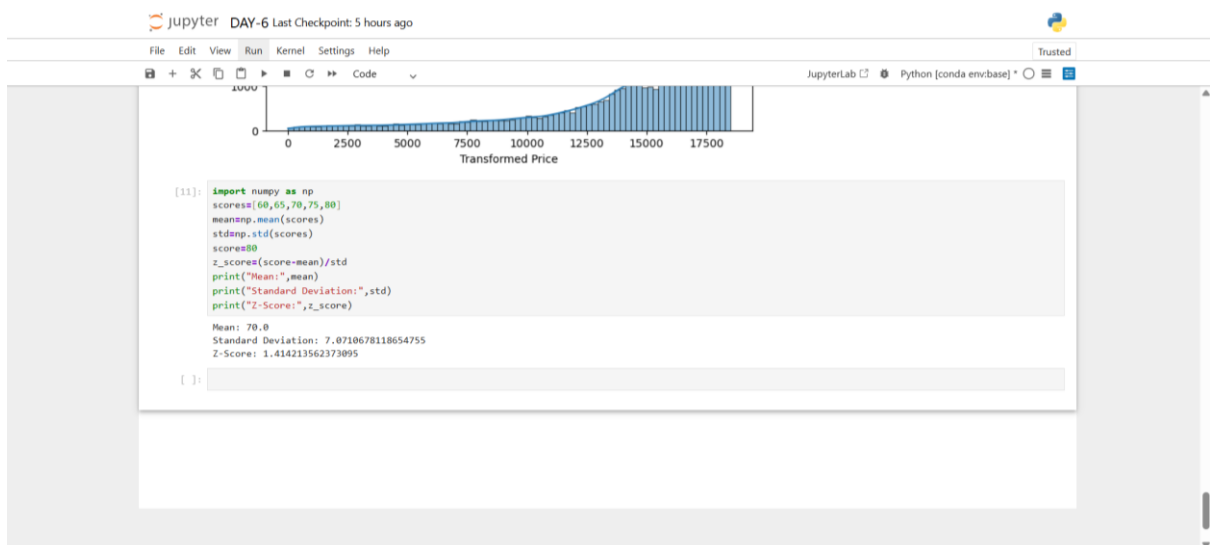
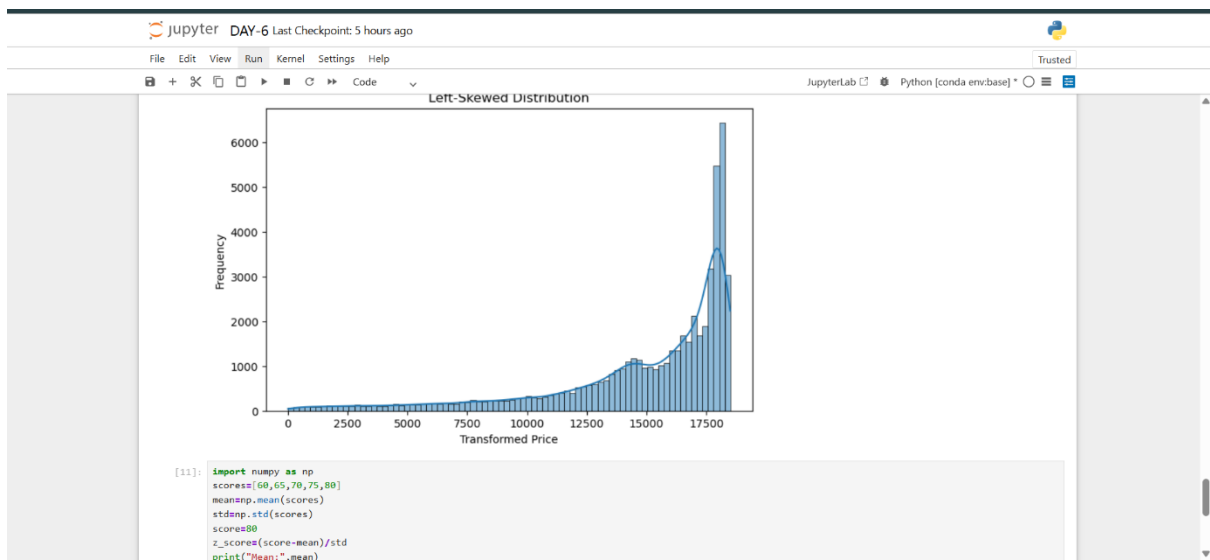
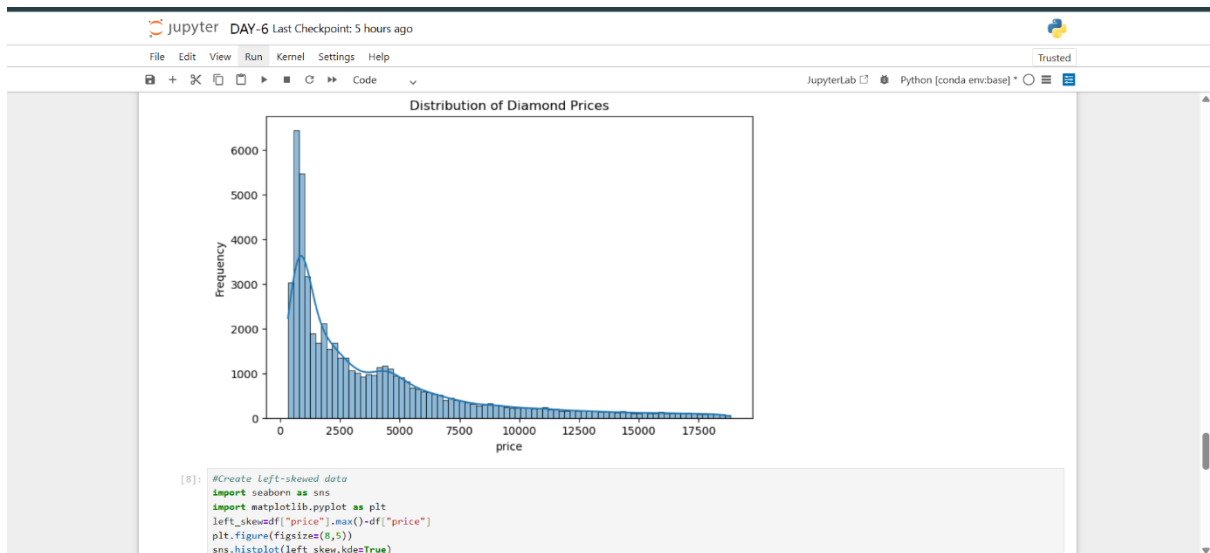
Pearson correlation: 0.986393238321439

[21]: #Spearman Correlation(monotonic)
import pandas as pd
dfapd.DataFrame([
    'Hours':[1,2,3,4,5],
    'Marks':[20,30,40,50,60]
])
corr=df['Hours'].corr(
    df['Marks'],
    method='spearman'
)
print("Spearman Correlation:",corr)

Spearman Correlation: 0.9999999999999999

[24]: #colleration matrix
# it contains both positive and negative relationship
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
dfapd.DataFrame([
    'Hours':[1,2,3,4,5],
    'Marks':[20,40,50,70,90],
    'Sleep':[8,7,6,5,4]
])
sns.heatmap(df.corr(),
            annot=True)
```





DAY-6 PRACTICE PROBLEM

```
jupyter DAY -6 Practice problem Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help Trusted
JupyterLab Python [conda env:base] * Anaconda Toolbox

[32]: import seaborn as sns

df = sns.load_dataset("tips")
print(df)
print(df.head())

total_bill  tip    sex smoker  day    time  size
0      16.99  1.01  Female    No  Sun  Dinner    2
1      10.34  1.66    Male    No  Sun  Dinner    3
2      21.01  3.50    Male    No  Sun  Dinner    3
3      23.68  3.31    Male    No  Sun  Dinner    2
4      24.59  3.61  Female    No  Sun  Dinner    4
...      ...    ...    ...    ...    ...    ...
239     29.03  5.92    Male    No  Sat  Dinner    3
240     27.18  2.00  Female   Yes  Sat  Dinner    2
241     22.67  2.00    Male   Yes  Sat  Dinner    2
242     17.82  1.75    Male    No  Sat  Dinner    2
243     18.78  3.00  Female    No  Thur Dinner    2

[244 rows x 7 columns]
total_bill  tip    sex smoker  day    time  size
0      16.99  1.01  Female    No  Sun  Dinner    2
1      10.34  1.66    Male    No  Sun  Dinner    3
2      21.01  3.50    Male    No  Sun  Dinner    3
3      23.68  3.31    Male    No  Sun  Dinner    2
4      24.59  3.61  Female    No  Sun  Dinner    4

[33]: print(df.columns)
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

```
jupyter DAY -6 Practice problem Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help Trusted
JupyterLab Python [conda env:base] * Anaconda Toolbox

[33]: print(df.columns)
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')

[34]: print(df.describe())

total_bill    tip    size
count  244.000000  244.000000  244.000000
mean    19.785943   2.998279   2.569672
std     8.902412   1.383638   0.951100
min     3.070000   1.000000   1.000000
25%    13.347500   2.000000   2.000000
50%    17.795000   2.900000   2.000000
75%    24.127500   3.562500   3.000000
max     50.810000  10.000000   6.000000

[35]: print(df.skew(numeric_only=True))
total_bill    1.133213
tip           1.465451
size          1.447882
dtype: float64

[36]: print(df["total_bill"].skew())
1.1332130376158205

[37]: print(df["tip"].skew())
1.4654510370979401

[38]: import matplotlib.pyplot as plt

plt.hist(df["total_bill"], bins=20)
plt.title("Distribution of Total Bill")
plt.xlabel("Total Bill")
```

